

1 Problem Assessment

This project aims to tackle an engineering optimization problem for a minimum-mass cantilevered box beam of length L , with an applied tip load, P . This structural optimization problem is a common engineering design challenge which is extremely relevant for the aerospace field as structural weight is always at a premium for air and spacecraft. Every unnecessary pound of weight results in exponentially decreasing performance. Therefore, it is of paramount importance that all non-essential weight is eliminated via a systematic process, while preserving necessary structural capability. For this particular problem, the material used is Aluminum 6061 T6, which is a high strength aluminum alloy commonly used for aerospace grade components. For this problem, the material properties, beam length, and tip force are shown in the Table 1.

Parameter	Value
ρ	2700 kg/m^3
L	10 m
E	$68.9 \times 10^9 \text{ Pa}$
σ_y	$276 \times 10^6 \text{ Pa}$
P	-50000 N

Table 1: Design Parameters

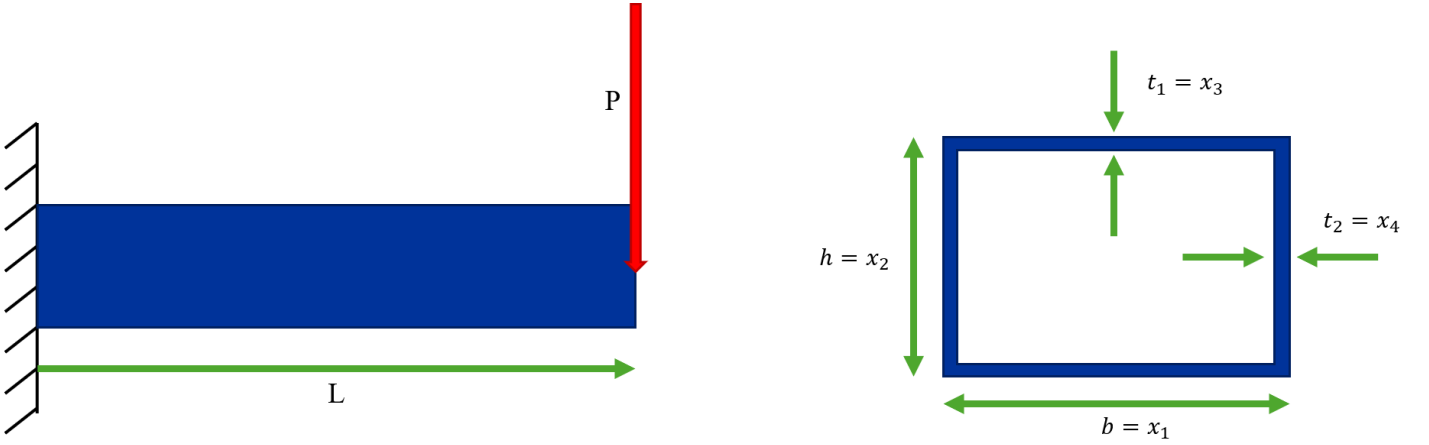


Figure 1: Problem Design

2 Equations and Models to be Used

2.1 Optimization Problem Formulation

The general formulation of a constrained optimization problem can be expressed as such:

$$\begin{aligned}
 &\underset{\mathbf{x} \in \mathbf{R}^n}{\text{minimize}} && f(\mathbf{x}) \\
 &\text{subject to} && \mathbf{g}(\mathbf{x}) = 0, \\
 & && \mathbf{h}(\mathbf{x}) \leq 0
 \end{aligned} \tag{1}$$

In the general case, $f(\mathbf{x})$ is the objective function that is minimized. $\mathbf{x}$ is the n -dimensional vector of design variables. $\mathbf{h}(\mathbf{x})$ and $\mathbf{g}(\mathbf{x})$ are the vectors of equality and inequality constraints that are enforced on the problem. In the context of this project, the optimization formulation can be refined:

$$\begin{aligned} & \underset{\mathbf{x} \in \mathbf{R}^4}{\text{minimize}} && f(\mathbf{x}) = m(\mathbf{x}) = \rho L A_c(\mathbf{x}) \\ & \text{subject to} && \mathbf{h}(\mathbf{x}) \leq 0 \end{aligned} \quad (2)$$

$$A_c(\mathbf{x}) = [(x_2)(x_1)) - ((x_2 - 2x_3)(x_1 - 2x_4))] \quad (3)$$

The objective function, $f(\mathbf{x})$ computes the mass, $m(\mathbf{x})$ of a cantilevered box beam as a function of the material density, ρ ($\frac{kg}{m^3}$), the beam length L (in meters), and the cross-sectional area, A_c (in m^2), which is itself a function of the design variable vector, $\mathbf{x}$. The vector of design variables, $\mathbf{x}$, is specified as:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} \quad (4)$$

where x_1 is the width of the cross section, x_2 is the height of the cross section, x_3 is the thickness of the top and bottom walls, and x_4 is the thickness of the side walls. All dimensions will be in meters for the purposes of this project. There are no hard equality constraints applied to this problem so as to make the feasible design space less restricted. For practical manufacturing considerations and to avoid non-zero cross-sectional dimensions, the following inequality constraints are applied:

$$\begin{aligned} h_1 &= x_2 - 1 \leq 0, & h_2 &= 0.1 - x_2 \leq 0, & h_3 &= -\delta_L(\mathbf{x}) - 0.5 \leq 0, & h_4 &= x_1 - 1 \leq 0 \\ h_5 &= 0.1 - x_1 \leq 0, & h_6 &= -1.5\sigma_{\max}(\mathbf{x}) - \sigma_{\text{yield}} \leq 0, & h_7 &= x_3 - 0.25 \leq 0 \\ h_8 &= 0.01 - x_3 \leq 0, & h_9 &= x_4 - 0.25 \leq 0, & h_{10} &= 0.01 - x_4 \leq 0 \end{aligned}$$

There are two functions of the design variable vector $\mathbf{x}$, which are the tip deflection, δ_L and the max bending stress of the beam, $\sigma_{\max}$. These are common analytic equations which eliminate the need for complex finite element modeling, making this optimization problem suitable for a class project level. These functions are given below in terms of the design variables:

$$\delta_L(\mathbf{x}) = \frac{PL^3}{3EI_{zz}(\mathbf{x})} = \frac{PL^3}{3E[\frac{1}{12}x_1x_2^3 - \frac{1}{12}(x_1 - 2x_4)(x_2 - 2x_3)^3]} \quad (5)$$

$$\sigma_{\max}(\mathbf{x}) = \frac{My_{\max}}{I_{zz}(\mathbf{x})} = \frac{(PL)(\frac{x_2}{2})}{[\frac{1}{12}x_1x_2^3 - \frac{1}{12}(x_1 - 2x_4)(x_2 - 2x_3)^3]} \quad (6)$$

In these functions, E is the Young's Modulus of the material of the beam, which will be assumed to be an linear, homogeneous, isotropic material and σ_{yield} is the yield strength of the material.

2.2 Numerical Methods

2.2.1 Sequential Quadratic Programming (SQP)

For this nonlinear inequality-constrained optimization problem, we will implement SQP to produce the minimum mass design for the cantilevered beam. SQP uses the method of Lagrange multipliers to find the constrained minimum of a function [1]. This process starts with defining the Lagrangian:

$$\mathcal{L}(\mathbf{x}, \lambda) = f(\mathbf{x}) + \lambda^T \mathbf{h}(\mathbf{x}) \quad (7)$$

where $f(\mathbf{x})$ is the objective function, λ^T is the vector of Lagrange multipliers, and $\mathbf{h}(\mathbf{x})$ is the vector of inequality constraints. Note that for this particular formulation, the Lagrange multipliers must be positive such that $\lambda_i > 0$ for $i = 1, 2, \dots, p$, where p is the total number of inequality constraints. Using both first & second

order Taylor series expansions, the nonlinear problem can be expressed as a quadratic subproblem at iteration k , where the objective function is quadratic and the constraints are linear. For a quadratic subproblem, enforcing the Karush-Kuhn-Tucker (KKT) optimality conditions generates the following linear system [2]:

$$\begin{bmatrix} [\nabla_{\mathbf{x}\mathbf{x}}\mathcal{L}_k] & [\nabla\mathbf{h}_{\mathcal{A}k}]^T \\ [\nabla\mathbf{h}_{\mathcal{A}k}] & [0] \end{bmatrix} \begin{pmatrix} s_x \\ s_\lambda \end{pmatrix} = \begin{pmatrix} -\nabla f(\mathbf{x}_k) \\ -\mathbf{h}_{\mathcal{A}}(\mathbf{x}_k) \end{pmatrix} \quad (8)$$

where the coefficient matrix is composed of the Hessian of the Lagrangian, along with the gradient vector of the active inequality constraints denoted by the subscript $\mathcal{A}$ [1]. Since this is an inequality constrained optimization, an active set method will be implemented to select the currently active inequality constraints for each local quadratic subproblem [1]. The solution of the linear system given by (8) provides step directions for both the design variable vector and the current Lagrange multiplier vectors [2], such that:

$$\begin{bmatrix} \mathbf{x}_{k+1} \\ \boldsymbol{\lambda}_{k+1} \end{bmatrix} = \begin{bmatrix} \mathbf{x}_k + s_x \\ \boldsymbol{\lambda}_k + s_\lambda \end{bmatrix} \quad (9)$$

Iteration will continue until the relative approximate error, ϵ_a is less than a set tolerance. Second and first derivatives will be computed analytically since the objective and constraint functions are relatively simple.

2.2.2 Logarithmic Barrier Method

In this study, the Logarithmic Barrier Method will be used as a way to obtain the design variables that meet the inequality constraints stated above. The formulation for this method is shown below where $f(\vec{x})$, is the objective function for mass and $h(\vec{x})$ is the vector of inequality constraints.

$$\phi(\vec{x}) = f(\vec{x}) - \mu \sum_{i=1}^q \log(-h_i(\vec{x})) \quad (10)$$

The key to this method is manually assigning μ so that this formulation converges effectively. A typical practice is to continually increase μ and record the converged values until a tolerance is met. This will be done via a convergence study to demonstrate that a sufficient value of μ has been met. To obtain the solution set for the Barrier Method, the above equation is minimized with respect to the design variables, which can be done using Newton's Method or the Barzilai-Borwein Method.

2.2.3 fmincon

For the purposes of this project, the MATLAB built-in function "fmincon" will be used as a way to find the minimum sizing constraints when given a set of physical constraints. The function is classified as a nonlinear programming solver by MATLAB and is used to find the minimum of given function, provided constraints and an initial guess within the bounds.

3 Implementation & Application of Numerical Methods

3.1 Sequential Quadratic Programming

Following the formulation outlined above, the SQP method was implemented in MATLAB. The objective function, its gradient, the constraint vector, and its Jacobian were all computed using the MATLAB Symbolic toolbox. All derivations were also completed in Mathematica as a check to make sure the optimizer was being fed the correct design problem. Once the setup was complete, MATLAB's "quadprog" was used inside a loop to iteratively solve the local quadratic subproblem at each step using the KKT conditions at the current point. This built-in function offers a way to solve quadratic programming problems with linear constraints [7]. This was especially suited to this problem since the nonlinear constraints were linearized at each iteration using the Jacobian, which meant "quadprog" could be used to solve the optimization problem, even though some of the original constraints are nonlinear in nature. One interesting note is that the current SQP implementation makes

use of the BFGS algorithm to update the Hessian. As the derivations for the Hessian of the Lagrangian were being completed, it was apparent that the analytical Hessian was quickly becoming cumbersome to program into MATLAB. As such, a decision was made to utilize the BFGS algorithm to update the Hessian from an initial guess. As discussed in Chapter 6 of Heath, the starting approximation used the identity matrix, and the following algorithm was used to update the Hessian at each iteration:

$$\mathbf{H}_{k+1} = \mathbf{H}_k + \frac{\mathbf{y}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} - \frac{\mathbf{H}_k (\mathbf{s}_k \mathbf{s}_k^T) \mathbf{H}_k}{\mathbf{s}_k^T \mathbf{H}_k \mathbf{s}_k} \quad (11)$$

Another thing to note is that initial runs of this algorithm were producing either infeasible results or converged to other local minima. This was fixed by implementing a curvature check for the BFGS update and ensuring that the Hessian update preserves the fact that $\mathbf{H}_k$ must be positive definite per [1], [5], and [6]. The curvature check is given by:

$$\mathbf{y}_k^T \mathbf{s}_k > 0 \quad (12)$$

Implementing this helped drive the SQP solver towards feasible local minima. Additionally, after solving the local KKT system for the quadratic subproblem using "quadprog", the resulting step direction was used to update the design variables using the following equation:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha \mathbf{s}_x \quad (13)$$

For the current implementation, α was manually iterated upon until the solver produced results that appeared reasonable. This was also a suggestion provided by Dr. Wang as a good initial implementation. This is something that can potentially be improved upon as most SQP methods use a line search to actually determine the correct step size to take at each iteration.

3.2 Logarithmic Barrier Method

Using the basic outline of the barrier method code that was written for Homework #8, this specific problem could be implemented. Newton's Method was chosen as the desired method which requires both the gradient and hessian of Equation (10). Seeing that there are 10 total constraints in this problem, the complexity of both the gradient and the hessian becomes much larger than the problem that was performed on Homework #8. To account for this complexity, the symbolic toolbox in MATLAB was used to ensure that both complex equations are correct based on the provided constraints. This study was ran with $\mu = [0.0001, 0.001, 0.01, 0.1, 1, 10, 100]$ to obtain not only a visual to see the effects of μ on the system, but to also determine the μ that provided the most accurate set of design variables. This code was the ran with smaller stepped μ values to get closer to the true solution.

3.3 MATLAB Built-In Function "fmincon"

The fmincon set up from homework 8 was used as the basis for the set up for the optimization problem. It was coded in 3 main sections, calling fmincon, defining the functions, and setting up the constraints. The problem features 10 constraints that are broken up into 2 sections in the code. The upper and lower bound constraints for b and h are in the section that calls fmincon as lb and ub. The rest, h_3, h_6-h_{10} are in the constraints function. The area function and the mass function are defined in the second section. An initial guess inside the constraints was chosen to be:

$$x_0 = \begin{bmatrix} 0.2 \\ 0.2 \\ 0.05 \\ 0.05 \end{bmatrix}$$

4 Initial Progress & Results

4.1 Sequential Quadratic Programming

Results for the SQP method are shown below. The algorithm used an initial guess that was within the size constraints on b, h, t_1 , and t_2 of:

$$\mathbf{x}_0 = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.1 \\ 0.1 \end{bmatrix} \quad (14)$$

For the current version of the method, α was set to 0.44.

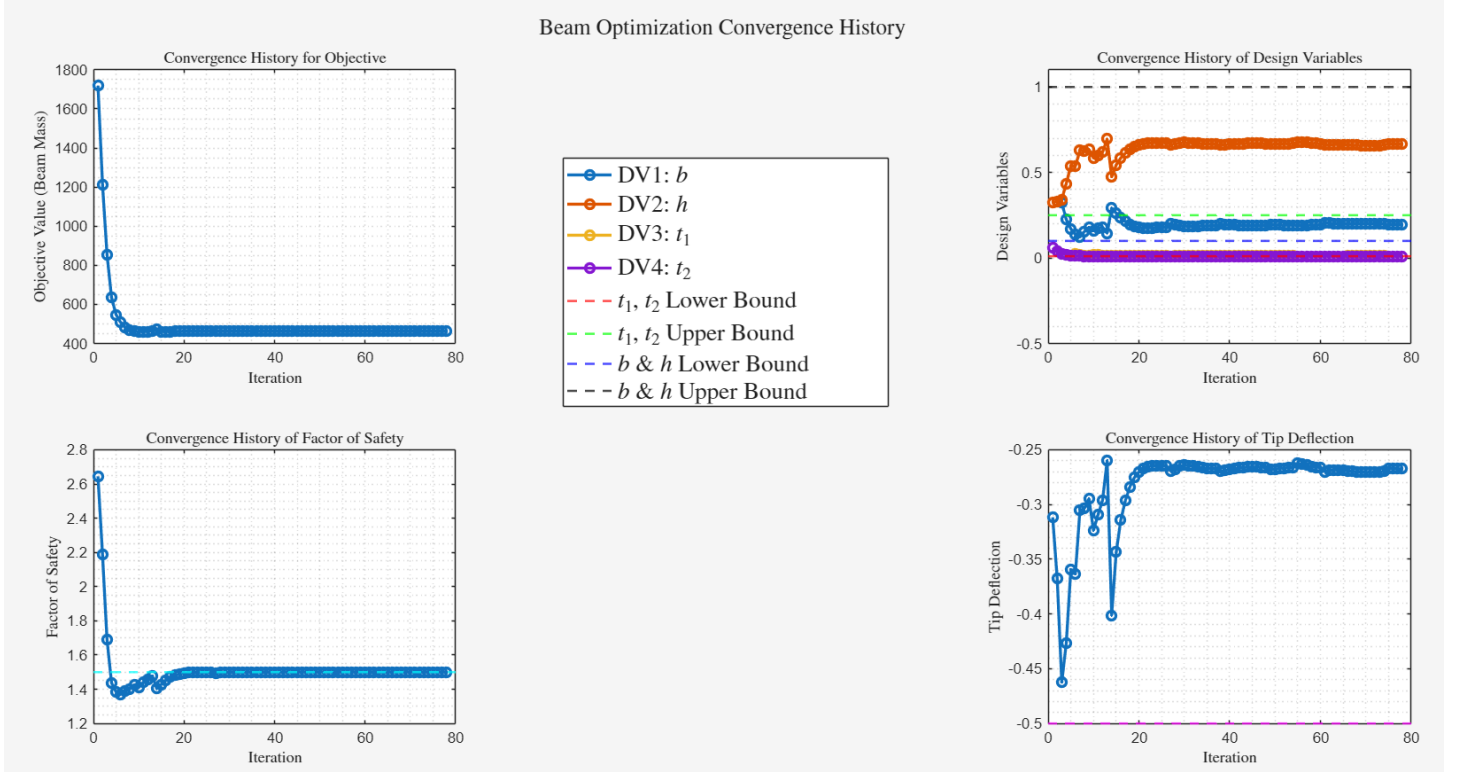


Figure 2: SQP Results for Beam Optimization Problem

The SQP method converged in 78 iterations. For the converged value, the resulting design variables are:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} b \\ h \\ t_1 \\ t_2 \end{bmatrix} = \begin{bmatrix} 0.1961 \\ 0.6658 \\ 0.0109 \\ 0.0100 \end{bmatrix} \quad (15)$$

The converged design variables result in an optimized beam mass of 468.4573 kilograms. At this solution, several constraints are active. Specifically, the lower bound on t_2 and the stress constraint are exactly active at the converged solution. This tells us that the optimized mass beam design may prefer to hit the stress limit instead of the deflection limit. Also, we note that the cross section of the beam has changed drastically to what might actually be expected for the problem. Starting from an initial guess with a square cross-section, the solver drives the design toward a beam that is narrow and tall as b is much less than h . From a physical sense, this makes sense since the resulting cross section maximizes the section moment of inertia, I_{zz} since that scales with h^3 . This minimizes the bending stress since I_{zz} is in the denominator. Driving the thicknesses close to the boundary is another effective way to minimize the mass since that thickness carries for the entire length of the beam.

4.2 Logarithmic Barrier Method

Implementing and running the barrier method with values of $\mu = [0.0001, 0.001, 0.01, 0.1, 1, 10, 100]$, the initial design variables could be determined and the objective function, the mass of the beam, could be used to determine the initial local minimum, which is shown in Figure 3. This was determined to be at $\mu = 0.1$ and corresponds to a mass of 463 kg.

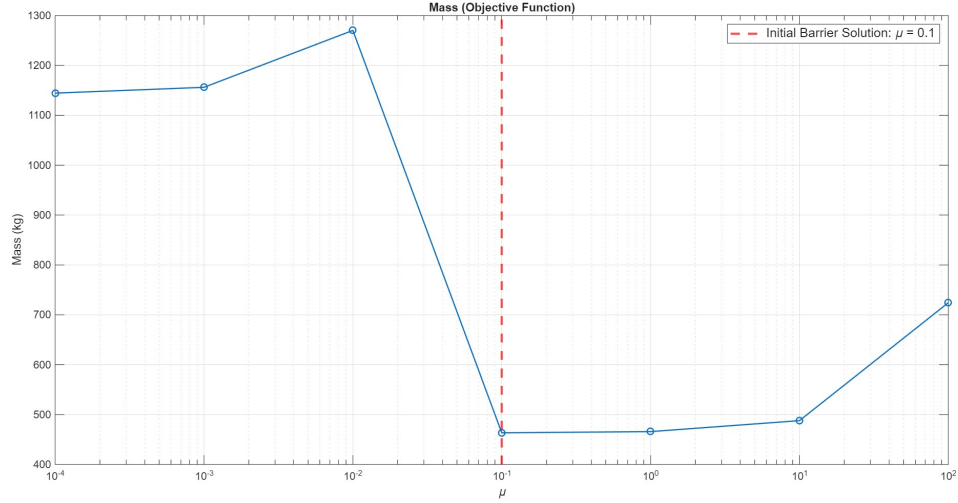


Figure 3: Objective Function Values for Increasing μ (Initial)

The design variables, shown in Figure 4, show that correlating the local mass minima results in the following dimensions of the beam which include $h = 0.6580$ m, $b = 0.2143$ m, $t_1 = 0.0103$ m, and $t_2 = 0.0100$ m.

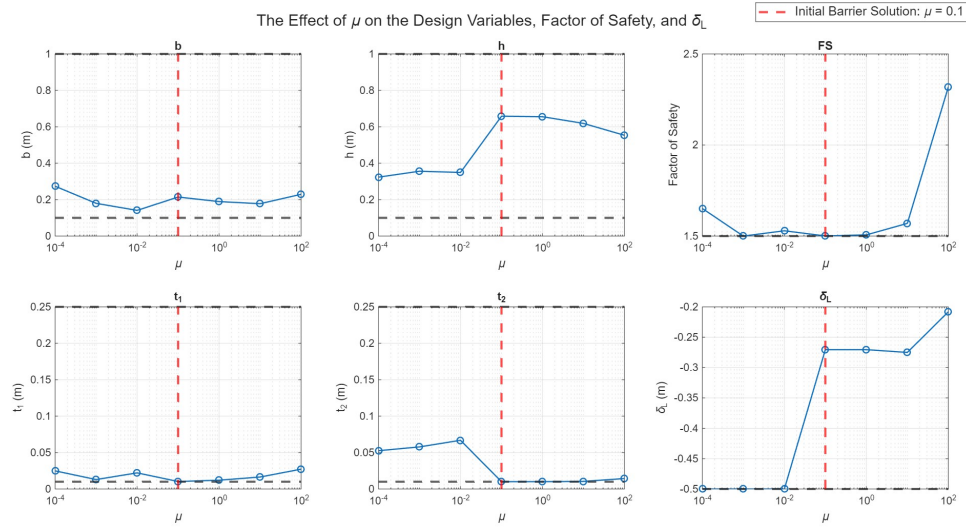


Figure 4: Initial Barrier Method Convergence with Respect to μ

Using these values of the initial barrier method solution, the deflection of the beam along the length is displayed in Figure 5.

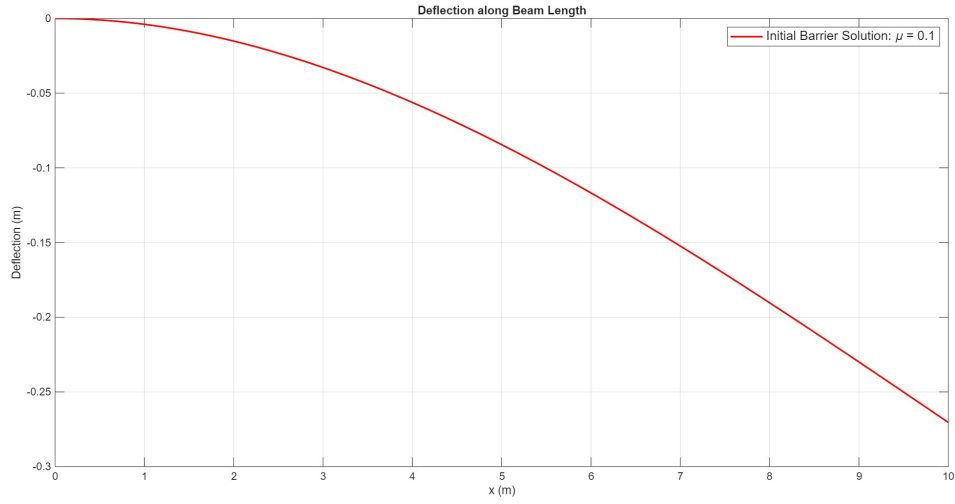


Figure 5: Deflection of Beam with Initial Barrier Solution

To further refine the optimum, the values of μ were refined into a smaller interval that encompasses the numbers $\mu = [0.01, 0.05, 0.1, 0.5, 1]$. The graph showing the final values of objective function for this updated barrier solution is shown in Figure 6.

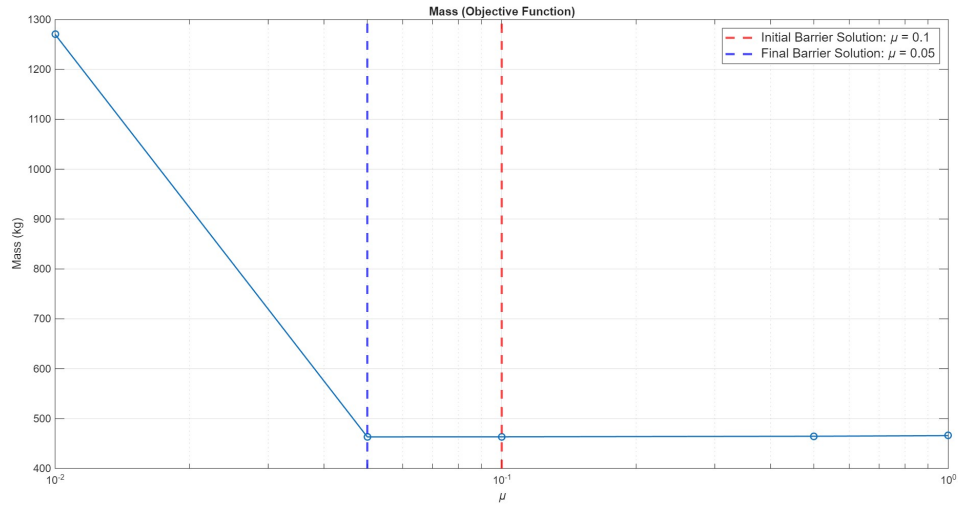


Figure 6: Objective Function Values for Increasing μ (Final)

The updated local minimum was found to be at $\mu = 0.05$, which decreased the total mass of the beam by 0.14 kg, which is a very small change compared to the entire mass of the beam which is 463.28 kg. From this small decrease, further refinement of this method is not necessary in this is an accurate approximation of the local minimum. The design variables for this updated solution are shown in Figure 7, which shows that the local mass minimum results in the following dimensions of the beam which include $h = 0.6580$ m, $b = 0.2168$ m, $t_1 = 0.0101$ m, and $t_2 = 0.0100$ m. It is important to note that even though b increased slightly, t_1 decreased.

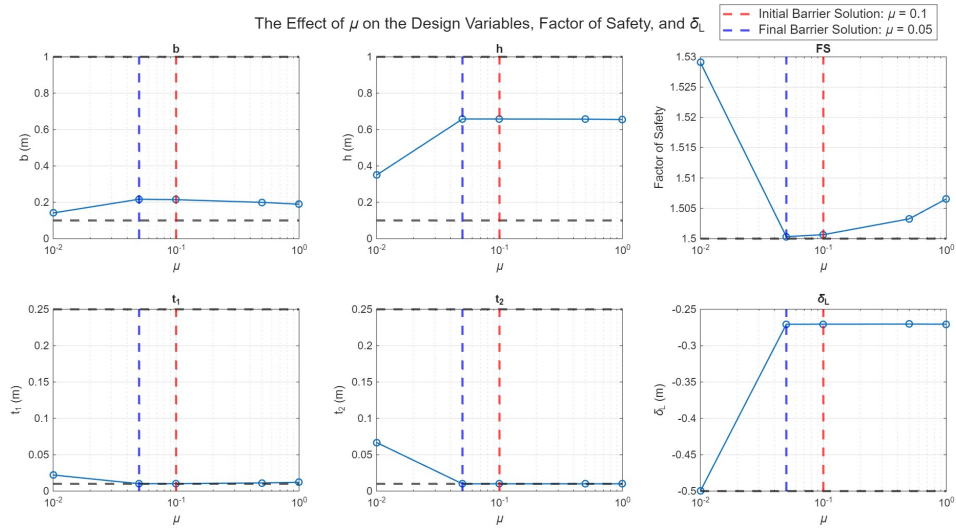


Figure 7: Final Barrier Method Convergence with Respect to μ

Another point to highlight is that the deflection shape or magnitude of the beam did not have any noticeable changes when updating the solution set which is shown in Figure 9.

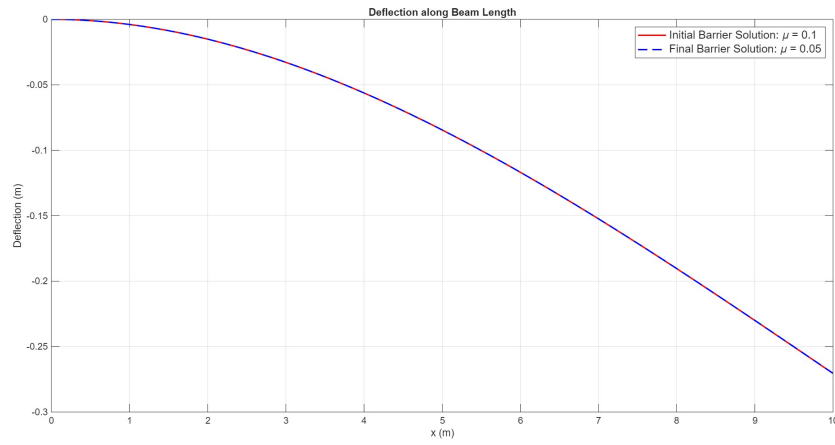


Figure 8: Deflection of Beam with Final Barrier Solution

4.3 MATLAB Built-In Function "fmincon"

The following results were found with the fmincon function for the described beam optimization problem. The found shape is a rectangle with a longer height than width

Parameter	Value
b	0.2193 m
h	0.6581 m
t_1	0.01 m
t_2	0.01 m
Mass	463.1438 kg
Iteration Count	15

Table 2: fmincon Results

Based on these results, the cross-section view of the beam would look like this:

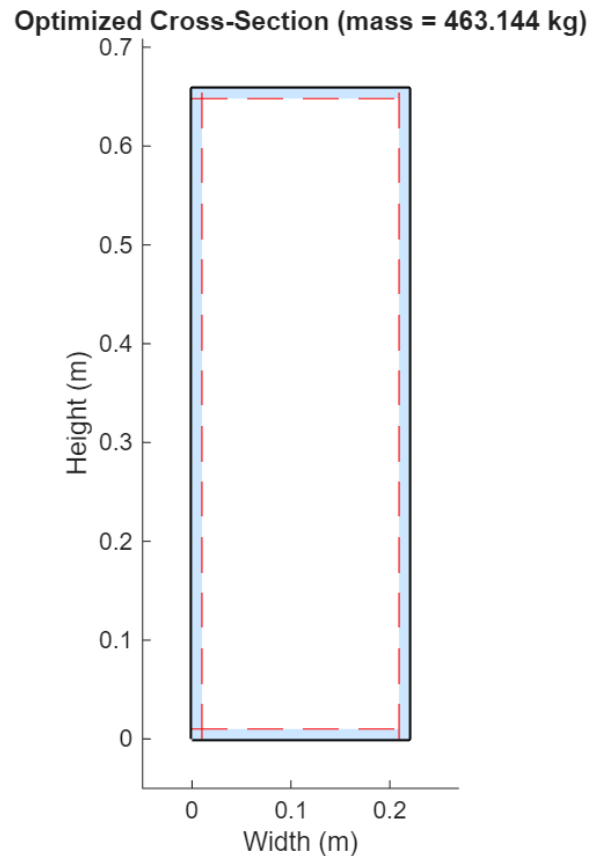


Figure 9: Optimized Cross-Sectional View

5 Work Completion Schedule

Each team member has made an attempt at solving the optimization problem, with the three methods described above. Joel is working on SQP, Ayden is working on Barrier method, and Madisyn is working on fmincon. Each team member is at the point where the code is functional and producing numbers very close to each other. Because of this, it can be assumed that all three discussed methods have been implemented correctly with small difference in values. Comparison of both numerical methods to fmincon will be conducted to show the percent error of each method in the final report. Additionally, we plan to run performance analysis on the methods as another point of comparison such as a rate of convergence test to see what method is the most efficient, along with time to convergence.

References

1. Health, M. T., Scientific Computing An Introductory Survey, SIAM, 2018.
2. Ning, "Equality-Constrained SQP" - BYU Flow Lab
3. MATLAB Help Center - Constrained Nonlinear Optimization Algorithms
4. MATLAB Help Center- fmincon
5. Lecture 11: Quasi-Newton: The BFGS and SR1 Methods
6. BFGS Algorithm Explained
7. MATLAB Help Center - quadprog

A Appendix

A.1 SQP

Listing 1: SQP Project Code

```
1 % Name: Joel George
2 % Date: 4/23/26
3 % This script is our group's attempt at formulating and implementing an SQP
4 % method to solve the nonlinear optimization problem
5 % of our project, which seeks to minimize the mass of a cantilvered beam
6 % with a tip load, subject to inequality constraints on the dimensions of
7 % the beam cross section and stress/deflection constraints.
8
9
10 %% Setting up Script:
11 clear; clc; close all;
12
13 %% Model Geometry, Properties, and Loads
14
15 % Geometry Inputs:
16 L_beam = 10; % cantilevered beam length, meters
17
18 % Material Properties (Al-6061 T6)
19 rho_beam = 2700; % density, kg/m^3
20 E_beam = 68.9e9; % Young's Modulus, Pa
21 sigma_yield = 276e6; % Tensile Yield Strength, Pa
22 nu_beam = 0.33; % Poisson's Ratio
23
24 % Loads:
25 P_applied = -50000; % applied Tip Load, Newtons
26
27 %% Defining Governing Equations:
28 x_sym = sym('x', [4 1]);
29
30 % Objective Function
31 f_sym = rho_beam*L_beam*( (x_sym(1))*(x_sym(2)) - (x_sym(1) - 2*x_sym(4))*(x_sym(2) -
    2*x_sym(3)) );
32 f = matlabFunction(f_sym, 'Vars', {x_sym});
33
34 % Gradient of Objective
35 gradf_sym = gradient(f_sym, x_sym);
36 gradf = matlabFunction(gradf_sym, 'Vars', {x_sym});
37
38 % Constraint Vector
39 Izz_sym = (1/12)*(x_sym(1)*x_sym(2)^3) - (1/12)*((x_sym(1) - 2*x_sym(4)) * (x_sym(2)
    - 2*x_sym(3)) ^3);
40 h_sym = [0.1 - x_sym(1);
    x_sym(1) - 1;
    0.1 - x_sym(2);
    x_sym(2) - 1;
    0.01 - x_sym(3);
    x_sym(3) - 0.25;
    0.01 - x_sym(4);
    x_sym(4) - 0.25;
    -(P_applied*L_beam^3)/(3*E_beam*Izz_sym) - 0.5;
    -1.5*(P_applied*L_beam*x_sym(2)/2)/(Izz_sym) - sigma_yield];
41 h = matlabFunction(h_sym, 'Vars', {x_sym});
42
43
44
45
46
47
48
49
50
51
```

```

52 % Constraint Jacobian
53 J_h_sym = jacobian(h_sym, x_sym);
54 J_h = matlabFunction(J_h_sym, 'Vars', {x_sym});
55
56 % Formulate Lagrange Multipliers and Lagrangian
57 lambda_sym = sym('lambda', [10 1]);
58 L_sym = f_sym + lambda_sym.' * h_sym;
59
60 % Gradient of Lagrangian
61 gradL_sym = gradient(L_sym, x_sym);
62 gradL = matlabFunction(gradL_sym, 'Vars', {x_sym, lambda_sym});
63
64
65 %% Initializing SQP solver:
66 x0 = [0.5; 0.5; 0.1; 0.1]; % initial guess for sizing
67 lambda0 = zeros(10,1); % initial guess of Lagrange Multitpliers
68 HessMat0 = eye(length(x0)); % initial guess for Hessian Matrix
69 solTol = 1e-6; % solution tolerance
70 maxIter = 1000; % maximum iteration count
71
72 %% SQP Algorithm - Active Set Method with BFGS Hessian Update
73
74 x_current = x0; % set initial guess for design variables
75 lambda_current = lambda0; % set initial guess for Lagrange Multipliers
76 HessMat = HessMat0; % set initial guess for the Hessian of the Lagrangian (this gets
    updated using BFGS)
77
78 history.dv_vals = [];
79 history.objval = [];
80 history.activeCons = cell(1,maxIter);
81 history.lambda = cell(1,maxIter);
82 history.iter = [];
83 qp_options = optimoptions('quadprog', 'Algorithm','active-set','Display','iter','
    MaxIterations',1000);
84
85 for k = 1:maxIter % begin loop
86
87     % Evaluate objective gradient, constraints, and constraint Jacobian at current
    point
88     gradf_current = gradf(x_current);
89     h_current = h(x_current);
90     J_h_current = J_h(x_current);
91
92     % Setup for quadprog
93     x0_quadprog = zeros(length(x_current), 1); % start with 0
94     H_quadprog = (HessMat + HessMat')/2; % ensure the Hessian is symmetric
95     f_quadprog = gradf_current; % pass in linear objective term
96     A_quadprog = J_h_current; % pass in linearized inequality constraints
97     b_quadprog = -h_current; % pass in linearized inequality constraints
98
99     % Solve the QP subproblem using quadprog
100     [dvStep, ~, exitflag, output, lambda_out] = quadprog(H_quadprog, f_quadprog,
        A_quadprog, b_quadprog, [], [], [], [], x0_quadprog, qp_options);
101     lambda_new = lambda_out.ineqlin;
102
103     % Advice from Dr. Wang - use a small step size along the calculated step direction
104     alpha = 0.44; % tune this to provide a suitable solution - should really be a line
    search
105     % alpha = 1.0; % base case - produced weird results

```

```

106 x_new = x_current + alpha*dvStep;
107
108 % Similar Feasibility Check as HW8's Barrier Method
109 % alpha = 1.0;
110 % x_new = x_current + alpha*dvStep;
111 % maxLS_iter = 75;
112 % LS_iter = 0;
113 % while any(h(x_new) > 0) && LS_iter < maxLS_iter % this is outside the feasible
    region
114 %     alpha = alpha*0.5;
115 %     x_new = x_current + alpha*dvStep; % Update the guess again
116 %     LS_iter = LS_iter + 1;
117 %     if LS_iter == maxLS_iter
118 %         disp('Line search did not converge within max iterations.')
119 %     else
120 %         % nothing
121 %     end
122 % end
123
124 % Update the Hessian using BFGS (Based on Algorithm 6.5 from Heath)
125 y_current = gradL(x_new, lambda_new) - gradL(x_current, lambda_current); %
    Gradient difference
126 s_current = x_new - x_current; % Step difference
127
128 % BFGS Curvature Check
129 if y_current' * s_current > 1e-6
130     HessMat = HessMat + (y_current * y_current') / (y_current' * s_current) - (
        HessMat * (s_current * s_current') * HessMat) / (s_current' * HessMat *
        s_current);
131 else
132     % keep the current Hessian since the curvature is negative
133 end
134
135 % Update design variable guess and Lagrange multiplier guess
136 x_current = x_new; % update the design variables
137 lambda_current = lambda_new; % update Lagrange Multipliers
138 objval = f(x_current); % compute the mass of the beam
139
140 % Identifying the active constraints
141 h_active_idx = find(lambda_current > 1e-5);
142
143 % Computing Tip Deflections and Factor of Safety
144 tipDef = (P_applied*L_beam^3)/( 3*E_beam*( (1/12)*x_current(1)*x_current(2)^3 -
    (1/12)*(x_current(1) - 2*x_current(4))*(x_current(2) - 2*x_current(3))^3 ) );
145 FoS = sigma_yield/( (-P_applied*L_beam*x_current(2)/2)/(( (1/12)*x_current(1)*
    x_current(2)^3 - (1/12)*(x_current(1) - 2*x_current(4))*(x_current(2) - 2*
    x_current(3))^3 ) ) );
146
147 % Store values and active constraints
148 history.dv_vals(:, k) = x_current; % Store the current design variable guess
149 history.lambda{k} = lambda_current; % Store the current Lagrange multipliers
150 history.objval(:,k) = objval; % Store the current beam mass
151 history.activeCons{k} = h_active_idx; % store the current active constraints
152 history.tipDef(:, k) = tipDef; % Store the current tip deflection
153 history.FoS(:,k) = FoS;
154 history.iter(k) = k; % Store the current iteration number
155
156 % Check for convergence
157 if norm(dvStep) < solTol

```

```

158         disp(['Converged in ', num2str(k), ' iterations.'])
159         break;
160     end
161
162 end
163
164 % Plotting the history of design variable values and objective function
165 figure(Theme="light")
166 sgtitle('Beam Optimization Convergence History','Interpreter','latex', 'FontWeight', '
    bold')
167 subplot(2,3,1);
168 plot(history.iter, history.objval, '-o', 'LineWidth',2);
169 grid minor
170 xlabel('Iteration','Interpreter','latex', 'FontWeight', 'bold');
171 ylabel('Objective Value (Beam Mass)','Interpreter','latex', 'FontWeight', 'bold');
172 title('Convergence History for Objective','Interpreter','latex', 'FontWeight', 'bold')
    ;
173
174 subplot(2,3,3);
175 plot(history.iter, history.dv_vals(1,:), '-o', 'LineWidth',2)
176 hold on; grid minor
177 plot(history.iter, history.dv_vals(2,:), '-o', 'LineWidth',2)
178 plot(history.iter, history.dv_vals(3,:), '-o', 'LineWidth',2)
179 plot(history.iter, history.dv_vals(4,:), '-o', 'LineWidth',2);
180
181 yline(0.01, 'r--', 'LineWidth',1.5)
182 yline(0.25, 'g--', 'LineWidth',1.5)
183 yline(0.1, 'b--', 'LineWidth',1.5)
184 yline(1, 'k--', 'LineWidth',1.5)
185
186 xlabel('Iteration','Interpreter','latex', 'FontWeight', 'bold');
187 ylabel('Design Variables','Interpreter','latex', 'FontWeight', 'bold');
188 ylim([-0.5 1.1])
189 title('Convergence History of Design Variables','Interpreter','latex', 'FontWeight', '
    bold');
190
191 subplot(2,3,4)
192 plot(history.iter, history.FoS, '-o', 'LineWidth',2)
193 grid minor
194 xlabel('Iteration','Interpreter','latex', 'FontWeight', 'bold');
195 ylabel('Factor of Safety','Interpreter','latex', 'FontWeight', 'bold');
196 yline(1.5, 'c--', 'LineWidth',1.5)
197 title('Convergence History of Factor of Safety','Interpreter','latex', 'FontWeight', '
    bold');
198
199 subplot(2,3,6)
200 plot(history.iter, history.tipDef, '-o', 'LineWidth',2)
201 grid minor
202 xlabel('Iteration','Interpreter','latex', 'FontWeight', 'bold');
203 ylabel('Tip Deflection','Interpreter','latex', 'FontWeight', 'bold');
204 yline(-0.5, 'm--', 'LineWidth',1.5)
205 title('Convergence History of Tip Deflection','Interpreter','latex', 'FontWeight', '
    bold');
206
207 subplot(2,3,3)
208 legend(["DV1:  $\$b\$$ ", "DV2:  $\$h\$$ ", "DV3:  $\$t_{1}\$$ ", "DV4:  $\$t_{2}\$$ ", " $\$t_{1}\$, \$t_{2}\$$  Lower
    Bound", " $\$t_{1}\$, \$t_{2}\$$  Upper Bound", " $\$b\$$  &  $\$h\$$  Lower Bound", " $\$b\$$  &  $\$h\$$  Upper
    Bound"], "Interpreter", "latex", "Location", "none", "Position", [0.4278 0.5577
    0.1488, 0.1659])

```

```

209 hLegend = findobj(gcf,"Type","legend");
210 hLegend.Interpreter = "latex";
211 hLegend.FontSize = 16;
212 hLegend.FontWeight = 'bold';

```

A.2 Logarithmic Barrier Method

Listing 2: Barrier Method Project Code

```

1  clc;
2  clear;
3  close all;
4
5  rho = 2700;
6  L = 10;
7  EE = 68.9e9;
8  %mu_vec = [0.0001;0.001;0.01;0.1;1;10;100];
9  mu_vec = [0.01;0.05;0.1;0.5;1];
10 sigma_y = 276e6;
11 P = -50000;
12
13 k_max = 5000;
14 k_vec = [];
15
16 x_k_vec_mu = zeros(4,length(mu_vec));
17
18 f = @(b,h,t1,t2) rho.*L.*((b.*h) - ((h- 2.*t1).*(b-2.*t2)));
19
20 syms b h t1 t2 mu
21
22 I = (b.*h.^3)./12 - ((h - 2.*t1).^3.*(b - 2.*t2))./12;
23
24 h_s = [
25 h - 1;
26 0.1 - h;
27 -0.5 - (L.^3.*P)./(3.*EE.*I);
28 b - 1;
29 0.1 - b;
30 -1.5*(h.*L.*P)./(2.*I) - sigma_y;
31 t1 - 0.25;
32 0.01 - t1;
33 t2 - 0.25;
34 0.01 - t2
35 ];
36
37 phi_s = (h*b - ((h - 2*t1)*(b-2*t2)))*rho*L - mu * sum(log(-h_s));
38
39 x = [b; h; t1; t2];
40 grad_s = gradient(phi_s, x);
41 hess_s = hessian(phi_s, x);
42
43 gradfunc = matlabFunction(grad_s, 'Vars', {b, h, t1, t2, mu});
44 hessfunc = matlabFunction(hess_s, 'Vars', {b, h, t1, t2, mu});
45 phifunc = matlabFunction(phi_s, 'Vars', {b, h, t1, t2, mu});
46 h_vec = matlabFunction(h_s, 'Vars', {b, h, t1, t2});
47
48
49 for m = 1:length(mu_vec)

```

```

50     mu_val = mu_vec(m);
51
52     x_k_NM = [0.51; 0.51; 0.051; 0.051];
53     x_k_vec_NM(:,1) = x_k_NM;
54     epsilon_a = 1000;
55     tol_for_problem = 10^-6;
56     k = 0;
57
58
59     while epsilon_a > tol_for_problem
60         H_k1_phi = hessfunc(x_k_NM(1),x_k_NM(2),x_k_NM(3),x_k_NM(4),mu_val);
61
62         subtracted_k1_NM = H_k1_phi\gradfunc(x_k_NM(1),x_k_NM(2),x_k_NM(3),x_k_NM(4),
63             mu_val);
64
65         alpha_NM = 1;
66
67         while any(h_vec(x_k_NM(1) - alpha_NM*subtracted_k1_NM(1), x_k_NM(2) - alpha_NM
68             *subtracted_k1_NM(2), x_k_NM(3) - alpha_NM*subtracted_k1_NM(3), x_k_NM(4)
69             - alpha_NM*subtracted_k1_NM(4)) > 0) && alpha_NM > 10^-14
70             alpha_NM = alpha_NM * 0.1;
71         end
72         x_k1_NM = x_k_NM - alpha_NM * subtracted_k1_NM;
73
74         epsilon_a = norm((x_k1_NM - x_k_NM))/norm(x_k1_NM);
75         x_k_NM = x_k1_NM;
76         x_k_vec_NM(:,k+2) = x_k_NM;
77         k = k + 1;
78     end
79
80     b_NM = x_k_NM(1);
81     h_NM = x_k_NM(2);
82     t1_NM = x_k_NM(3);
83     t2_NM = x_k_NM(4);
84
85     %% Constraint Variables
86
87     Izz = (1/12).*x_k_vec_mu(1,:).*x_k_vec_mu(2,:).^3 - (1/12).*(x_k_vec_mu(1,:) - 2.*
88         x_k_vec_mu(4,:)).*(x_k_vec_mu(2,:) - 2.*x_k_vec_mu(3,:)).^3;
89
90     f(b_NM,h_NM,t1_NM,t2_NM);
91
92     sigma_max = -(P*L*(x_k_vec_mu(2,:)/2))./Izz;
93
94     FS = abs(sigma_y./sigma_max);
95
96     delta_L = (P.*L.^3)./(3.*EE.*((1/12).*x_k_vec_mu(1,:).*x_k_vec_mu(2,:).^3 - (1/12).*(
97         x_k_vec_mu(1,:) - 2.*x_k_vec_mu(4,:)).*(x_k_vec_mu(2,:) - 2.*x_k_vec_mu(3,:)).^3));
98
99     fprintf('b = %4.2f, h = %4.2f, t_1 = %4.2f, t_2 = %4.2f using Newtons Method \n',
100         x_k_NM(1), x_k_NM(2), x_k_NM(3), x_k_NM(4));
101
102     %% Deflection
103     minima_val2 = 2;

```

```

103
104 deflection2 = @(x) -P*(3*L*x^2 - x^3)/(6*EE*Izz(minima_val2));
105 b_vals2 = x_k_vec_mu(1,minima_val2);
106 h_vals2 = x_k_vec_mu(2,minima_val2);
107 t1_vals2 = x_k_vec_mu(3,minima_val2);
108 t2_vals2 = x_k_vec_mu(4,minima_val2);
109
110 x_plot = linspace(0, L, 201);
111 defl_vals2 = P.*(3*L.*x_plot.^2 - x_plot.^3)./(6*EE*Izz(minima_val2));
112
113 minima_val3 = 3;
114
115 deflection3 = @(x) -P*(3*L*x^2 - x^3)/(6*EE*Izz(minima_val3));
116 b_vals3 = x_k_vec_mu(1,minima_val3);
117 h_vals3 = x_k_vec_mu(2,minima_val3);
118 t1_vals3 = x_k_vec_mu(3,minima_val3);
119 t2_vals3 = x_k_vec_mu(4,minima_val3);
120 defl_vals3 = P.*(3*L.*x_plot.^2 - x_plot.^3)./(6*EE*Izz(minima_val3));
121
122
123 p5 = plot(x_plot, defl_vals2, 'r-', 'LineWidth', 1.5);
124 hold on
125
126 p6 = plot(x_plot, defl_vals3, 'b--', 'LineWidth', 1.5);
127 grid on;
128 xlabel('x (m)');
129 ylabel('Deflection (m)');
130 title('Deflection along Beam Length');
131
132 legend([p5 p6],{'Initial Barrier Solution: \mu = 0.1','Final Barrier Solution: \mu = 0.05'},FontSize=12);
133 %% Objective Function
134
135 % evaluate f for each design (columns of x_k_vec_mu) and plot versus mu
136 f_vals = zeros(1, size(x_k_vec_mu,2));
137 for i = 1:size(x_k_vec_mu,2)
138     f_vals(i) = f(x_k_vec_mu(1,i), x_k_vec_mu(2,i), x_k_vec_mu(3,i), x_k_vec_mu(4,i))
139 end
140 semilogx(mu_vec, f_vals, '-o','LineWidth',1.2);
141 hold on
142 grid on;
143 xlabel('\mu');
144 ylabel('Mass (kg)');
145 title('Mass (Objective Function)')
146 p3 = xline(0.1,LineStyle='--',LineWidth=2,Color='r');
147 xlim([min(mu_vec), max(mu_vec)])
148 %p4 = xline(0.05,LineStyle='--',LineWidth=2,Color='b');
149 legend([p3],{'Initial Barrier Solution: \mu = 0.1','Final Barrier Solution: \mu = 0.05'},FontSize=12);
150 %% Plots
151 mu_plot = mu_vec;
152
153 sgtitle('The Effect of \mu on the Design Variables, Factor of Safety, and \delta_L')
154 subplot(2,3,1);
155 semilogx(mu_plot, x_k_vec_mu(1,:), '-o','LineWidth',1.2);
156 title('b')
157 hold on
158 yline(0.1,LineStyle='--',LineWidth=2)
159 yline(1,LineStyle='--',LineWidth=2)

```



```

160 p1 = xline(0.1,LineStyle='--',LineWidth=2,Color='r');
161 p2 = xline(0.05,LineStyle='--',LineWidth=2,Color='b');
162 grid on;
163 xlabel('\mu');
164 ylabel('b (m)');
165 xlim([min(mu_vec), max(mu_vec)])
166
167 subplot(2,3,2);
168 semilogx(mu_plot, x_k_vec_mu(2,:), '-o','LineWidth',1.2);
169 title('h')
170 hold on
171 yline(0.1,LineStyle='--',LineWidth=2)
172 yline(1,LineStyle='--',LineWidth=2)
173 xline(0.1,LineStyle='--',LineWidth=2,Color='r')
174 xline(0.05,LineStyle='--',LineWidth=2,Color='b');
175 grid on;
176 xlabel('\mu');
177 ylabel('h (m)');
178 xlim([min(mu_vec), max(mu_vec)])
179
180 subplot(2,3,4);
181 semilogx(mu_plot, x_k_vec_mu(3,:), '-o','LineWidth',1.2);
182 title('t_1')
183 hold on
184 yline(0.01,LineStyle='--',LineWidth=2)
185 yline(.25,LineStyle='--',LineWidth=2)
186 xline(0.1,LineStyle='--',LineWidth=2,Color='r')
187 xline(0.05,LineStyle='--',LineWidth=2,Color='b');
188 grid on;
189 xlabel('\mu');
190 ylabel('t_1 (m)');
191 xlim([min(mu_vec), max(mu_vec)])
192
193 subplot(2,3,5);
194 semilogx(mu_plot, x_k_vec_mu(4,:), '-o','LineWidth',1.2);
195 title('t_2')
196 hold on
197 yline(0.01,LineStyle='--',LineWidth=2)
198 yline(.25,LineStyle='--',LineWidth=2)
199 xline(0.1,LineStyle='--',LineWidth=2,Color='r')
200 xline(0.05,LineStyle='--',LineWidth=2,Color='b');
201 grid on;
202 xlabel('\mu');
203 ylabel('t_2 (m)');
204 xlim([min(mu_vec), max(mu_vec)])
205
206 subplot(2,3,3);
207 semilogx(mu_plot, FS, '-o','LineWidth',1.2);
208 title('FS')
209 hold on
210 yline(1.5,LineStyle='--',LineWidth=2)
211 xline(0.1,LineStyle='--',LineWidth=2,Color='r')
212 xline(0.05,LineStyle='--',LineWidth=2,Color='b');
213 grid on;
214 grid on;
215 xlabel('\mu');
216 ylabel('Factor of Safety');
217 xlim([min(mu_vec), max(mu_vec)])
218

```

```

219 subplot(2,3,6);
220 semilogx(mu_plot, delta_L, '-o','LineWidth',1.2);
221 title('\delta_L')
222 hold on
223 yline(-0.5,LineStyle='--',LineWidth=2)
224 xline(0.1,LineStyle='--',LineWidth=2,Color='r')
225 xline(0.05,LineStyle='--',LineWidth=2,Color='b');
226 grid on;
227 xlabel('\mu');
228 ylabel('\delta_L (m)');
229 xlim([min(mu_vec), max(mu_vec)])
230 legend([p1 p2],{'Initial Barrier Solution: \mu = 0.1','Final Barrier Solution: \mu =
0.05'},FontSize=12);

```

A.3 "fmincon

Listing 3: "fmincon" Project Code

```

1 %Numerical Methods Project
2 %fmincon method
3
4 %%
5 % %General Set up
6 % %Definitions
7 %
8 % %General Variables
9 % rho = 2700; %density
10 % L = 10; %total beam length
11 % EE = 68.9e9; %Young's Modulus of material
12 % sigma_y = 276e6; %yield strength of the material
13 % P = -50000; %Applied tip load
14 %
15 % %X=[width; height; thickness top and bottom; thickness side]
16 % %b= width of the cross section
17 % %h= height of the cross section
18 % %t1= thickness of the top and bottom
19 % %t2= thickness of the side walls
20 %
21 % %Equations
22 %
23 % A = @(b,h,t1,t2) ((h*b)-(h-2*t1)*(b-2*t2)); %Cross Sectional Area
24 %
25 % f = @(b,h,t1,t2) rho.*L.*A;
26 %
27 % delta_L= @(b,h,t1,t2) (P*L^3)/((3*EE*((1/12)*b*h^3)-(1/12)*(b-2*t2)*(h-2*t1)^3));
28 %
29 % sigma_max= @(b,h,t1,t2) -(P*L*(h/2))/(((1/12)*b*h^3)-(1/12)*(b-2*t2)*(h-2*t1)^3);
30
31 %%
32 %Call fmincon
33 clear
34 close all
35 clc
36 rho = 2700; %density
37 L = 10; %total beam length
38 EE = 68.9e9; %Young's Modulus of material
39 sigma_y = 276e6; %yield strength of the material
40 P = -50000; %Applied tip load

```

```

41
42 nonlcon = @constraints;
43 A = [];
44 b = [];
45 Aeq = [];
46 beq = [];
47 lb = [0.1, 0.1, 0.01, 0.01];
48 ub = [1, 1, 0.25, 0.25];
49 x0 = [.2, .2, 0.05, 0.05];
50 options = optimoptions('fmincon','Algorithm','interior-point',
    'SpecifyObjectiveGradient',false, ...
51     'SpecifyConstraintGradient',false,'MaxIterations',200,'OptimalityTolerance',1e-6,
    'Display','iter-detailed');
52 [x,fval]=fmincon(@(x) beam(x, rho, L), x0, A, b, Aeq, beq, lb, ub, @(x) constraints(x,
    P,L,EE,sigma_y), options);
53
54
55
56 %%
57 %Function Definitions
58
59 function F= beam(x, rho, L)
60
61 b=x(1);
62 h=x(2);
63 t1=x(3);
64 t2=x(4);
65
66 A=(b*h)-(b-2*t2)*(h-2*t1); %Cross Sectional Area
67
68 F = rho*L*A;
69
70 end
71
72 %%
73 %Constraints
74 function [c,ceq] = constraints(x,P,L,EE,sigma_y)
75
76 b=x(1);
77 h=x(2);
78 t1=x(3);
79 t2=x(4);
80
81 delta_L= (P*L^3) / ( 3*EE* ( ( (1/12)*b*h^3 ) -( (1/12)*(b-2*t2)*((h-2*t1)^3) ) ) );
82
83 sigma_max= -(P*L*(h/2)) / ( ((1/12)*b*h^3)-((1/12)*(b-2*t2)*(h-2*t1)^3) );
84
85 delta_max=0.5;
86
87 c=zeros(6,1);
88
89 c(1)=sigma_max-(sigma_y/1.5);
90
91 c(2)=-delta_L-delta_max;
92
93 c(3)=2*t2-b;
94
95 c(4)=2*t1-h;
96

```

```
97 c(5)=-t1;  
98  
99 c(6)=-t2;  
100  
101 ceq=[];  
102  
103 end
```